

Ejercicio Stack

PRÁCTICA DE ESTRUCTURAS DE DATOS: TDA PILA (`std::stack`)

Ejercicio de Programación en C++

Estructura Obligatoria

Para implementar la solución, tu programa deberá utilizar estrictamente el siguiente `struct`:

```
1 struct Nodo {  
2     std::string sustancia;  
3     int reactividad; // Positivo = Acido | Negativo = Alcalino  
4 };
```

Reglas

- 1. Ley de la Inercia:** Si la pila está vacía, o si el elemento en el `top()` tiene el **mismo signo** que el nuevo Nodo (ambos positivos o ambos negativos), el nuevo Nodo cae y se apila pacíficamente encima.
- 2. Ley de la Aniquilación Mutua:** Si el `top()` y el nuevo Nodo tienen signos opuestos, pero **exactamente el mismo valor absoluto** (ejemplo: un `+4` cayendo sobre un `-4`), ambos se neutralizan, se hacen humo y desaparecen. (*Debes hacer `pop()` al `top()` y el nuevo nodo no entra*).
- 3. Ley del Dominio (La regla de oro):** Si tienen signos opuestos y **distinto valor absoluto**, el más fuerte destruye al más débil, pero pierde tanta reactividad como la fuerza que tenía el destruido:
 - **Si el que ya estaba en el `top()` era más fuerte** (ej: `top = +7`, nuevo = `-3`), el nuevo se desintegra y el `top()` sobrevive, pero su reactividad baja a `+4`.
 - **Si el nuevo Nodo es el fuerte** (ej: `top = -2`, nuevo = `+5`), el nuevo destruye al `top()` (haces `pop()`), el nuevo Nodo reduce su fuerza a `+3` y **continúa cayendo** para intentar reaccionar con el siguiente elemento que haya quedado de `top()`.

Instrucciones

Escribir una función que reciba un `std::vector<Nodo>` con la secuencia de ingredientes que Ponciano deja caer, y devuelva la `std::stack<Nodo>` resultante con los ingredientes que sobrevivieron (y su carga neta final).

Caso de Prueba

Dada la siguiente secuencia de entrada exacta (en orden de índice):

```
1 std::vector<Nodo> ingredientes = {  
2     {"Plomo", 6},  
3     {"Magnesio", 3},  
4     {"Acido Suave", -3},  
5     {"Super-Base", -8},  
6     {"Sal de Mesa", 5}  
7 };
```

Ejecución paso a paso:

1. **Entra Plomo (+6)** → Pila: [Plomo(6)]
2. **Entra Magnesio (+3)** (*Mismo signo*) → Pila: [Plomo(6), Magnesio(3)]
3. **Entra Ácido Suave (-3)** → Choca con Magnesio (+3). **Ley 2** (Aniquilación). Se destruyen ambos.
Pila resultante: [Plomo(6)]
4. **Entra Super-Base (-8)** → Choca con Plomo (+6). **Ley 3**.
La Super-Base gana y destruye al Plomo (`pop()`). Pasa de fuerza -8 a -2 .
Como destruyó al Plomo, **sigue cayendo**. Al quedar la pila vacía, se asienta.
Pila resultante: [Super-Base(-2)]
5. **Entra Sal de Mesa (+5)** → Choca con Super-Base (-2). **Ley 3**.
La Sal de Mesa gana y destruye a la base (`pop()`). Pasa de fuerza $+5$ a $+3$. Se asienta.

ESTADO FINAL DE LA PILA:

[{"Sal de Mesa", 3}] (*Tamaño de la pila: 1*)